高级程序设计实验报告

实验八

题目:	实验八	指导教师:	冯国栋
姓名:	林楗棋	学号:	24363050
专业班号:	202513912	学院:	智能工程学院

目录

1 实验目的与设计思路 **2**

1.1 实验目的 2

1.2 设计思路 2

2 核心代码与测试结果 **2**

2.1 核心代码 2

2.1.1 utils 类的设置 2

2.1.2 重要代码 3

2.1.3 Print 函数群 3

2.2 测试结果 4

3 心得体会 **5**

4 加分项 **5**

5 附录：全部代码 **6**

实验目的与设计思路

1.1 实验目的

题目 1.1： 设计一个类 FileProcessing，利用 STL 数据结构实现对文本中单词的处理：

1. 将文本逐行读取，记录单词及其行列号；
2. 提供查询函数判断单词是否出现；
3. 统计给定单词出现的次数；
4. 增加新单词并保持位置合法；
5. 保存到文件，保留行列位置；
6. 删除给定单词，并将其后单词列号前移；
7. 显示单词及其行列；
8. 打印按字典序升序排列的单词；
9. 打印每个单词出现次数和高频词。

本实验主要训练对 **map/set/vector** 等容器的使用，掌握字符串处理与文件 IO，并通过类的封装完成一个小型文本索引系统。

1.2 设计思路

核心数据结构为 **std::map<std::string, std::vector<std::pair<int,int>>>**，其中键为单词，值为该单词在文本中出现的行列号列表。使用 **std::set** 维护出现过的单词集合。读取文本时按行扫描、按空格分词，逐个记录行号与列号。查询与统计直接基于 map 键值实现；新增单词时先检查位置合法性（避免重叠或跨行非法位置）；删除单词时需要移除该单词并对同一行其后单词进行列号前移，保证行内顺序连贯；排序输出时利用 **std::sort** 和 **std::string** 字典序比较。

核心代码与测试结果

2.1 核心代码

2.1.1 utils 类的设置

由于有些函数不属于 **Fileprocessing** 类的类函数，如文件读取函数，统计 **map** 中的字母的个数等，算是助手函数，可以放在 utils 文件中，但是 **utils** 也不是一个类，不能用 **class utils** 来声明。这种情况下可以用 **namespace** 来声明 **utils** 中的各种函数，比如

Listing 1: utils 声明

```cpp
#ifndef UTILS_H
#define UTILS_H
#include ...
namespace utils {
    void File2WordMap(const std::string& filename, std::map<std::string, std::vector<std::pair<int, int>>>& wordMap);
    ...
};
#endif
```

这样就可以在调用 **utils** 中的函数时仍然要声明其 namespace 但是不用创建一个函数了。

2.1.2 重要代码

本次实验中比较重要的有：逐行读取，增加字，删减字，和 Print 一系列函数。其中 Print 函数群单独讲，此处介绍前三个函数的实现逻辑。

1. 逐行读取使用 **ifstream** 逐行读取文本，通过 **getline**[1]获取整行，再用 **istringstream** 按空格分词；每读到一个单词，行号加一、列号自增，并将位置写入 **wordMap[word]**。
2. 增加字由于新增字不能重叠、不能跨行、前方需有字词，因此新增单词时先调用 **CheckViable** 检查行列是否合法，若合法则向 **wordMap[word]** 追加 **(row, col)**。
3. 删减字先统计被删词在每一行的列号集合；遍历所有单词位置时，若位置属于被删词则跳过，否则根据该行被删列号的数量将列号前移。

2.1.3 Print 函数群

四个 Print 函数都和顺序有一定关系，但是 **map** 本身没有顺序，所以需要使用 **utils** 中的 **getCount** 函数，将 **map** 转为 **vector**，vector 可以依靠 **std::sort**[2]来给 vector 类排序。但是如果遇到需要自定义排序顺序的地方则需要自定义排序函数，这时候可以用 **lambda** 表达式化简[3]。使输出的顺序符合函数要求的排序。

Listing 2: getCount 函数

```cpp
    std::vector<std::pair<std::string, int>> getCount(std::map<std::string, std::vector<std::pair<int, int>>>& wordMap) {
        std::vector<std::pair<std::string, int>> wordCount;
        for (auto entry : wordMap) {
            std::pair<std::string, int> pair = std::make_pair(entry.first, entry.second.size());
            wordCount.push_back(pair);
        }
```

[1]https://www.runoob.com/cplusplus/cpp-files-streams.html
[2]https://blog.csdn.net/weixin_41588502/article/details/86620305
[3]https://blog.csdn.net/m0_60134435/article/details/136151698

```
        std::sort(wordCount.begin(), wordCount.end(), [](const auto& a, const auto& b) {return a.second < b.second;});
        return wordCount;
    }
```

Listing 3: 统计与字典序输出

```
    void FileProcessing::PrintWordOccur() {
    std::vector<std::pair<std::string, int>> wordCount = utils::getCount(wordMap);
    for (auto& entry : wordCount) {
        std::cout << entry.first << " ";
    }
    std::cout << std::endl;
}
```

选做的打印前 **n** 个字也可以使用这个函数，求出依序下降的顺序，然后求 n 和 vector 大小的最小值，之后打印这个最小值项即可。

2.2 测试结果

对于测试部分，本实验采用自动化测试方案————**Google Test**[4]。Google Test 有简易的基础语法如 TEST()、EXPECT_EQ()、ASSERT_TRUE()，可以以此验证程序是否符合预期。
同时采用自动化测试，可以:

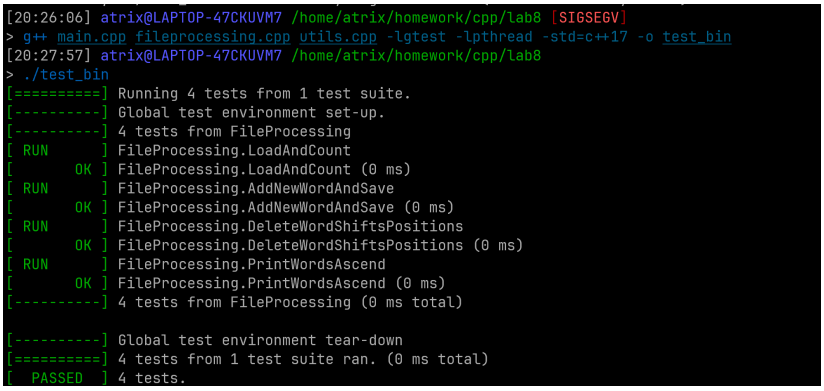
1. 自动检测程序逻辑错误，减少人工测试工作量；
2. 将测试与实现分离，提高程序的可维护性和可靠性。

本次测试用例分为了两个测试函数，分别测试初始化打印和增删查，思路为先从**.txt** 文件中读取数据[5]，然后进行相应操作并与预期结果对照验证正确性。主要覆盖:

1. 读取文本与出现次数统计；
2. 增加新词并保存到文件；
3. 删除词并检查列号前移；
4. 字典序打印输出。

得到测试结果为:

[4]https://google.github.io/googletest/primer.html
[5]https://www.runoob.com/cplusplus/cpp-files-streams.html



```
[20:26:06] atrix@LAPTOP-47CKUVM7 /home/atrix/homework/cpp/lab8 [SIGSEGV]
> g++ main.cpp fileprocessing.cpp utils.cpp -lgtest -lpthread -std=c++17 -o test_bin
[20:27:57] atrix@LAPTOP-47CKUVM7 /home/atrix/homework/cpp/lab8
> ./test_bin
[==========] Running 4 tests from 1 test suite.
[----------] Global test environment set-up.
[----------] 4 tests from FileProcessing
[ RUN      ] FileProcessing.LoadAndCount
[       OK ] FileProcessing.LoadAndCount (0 ms)
[ RUN      ] FileProcessing.AddNewWordAndSave
[       OK ] FileProcessing.AddNewWordAndSave (0 ms)
[ RUN      ] FileProcessing.DeleteWordShiftsPositions
[       OK ] FileProcessing.DeleteWordShiftsPositions (0 ms)
[ RUN      ] FileProcessing.PrintWordsAscend
[       OK ] FileProcessing.PrintWordsAscend (0 ms)
[----------] 4 tests from FileProcessing (0 ms total)

[----------] Global test environment tear-down
[==========] 4 tests from 1 test suite ran. (0 ms total)
[  PASSED  ] 4 tests.
```

图 1: 测试结果

可以认为程序符合预期，是正确的。

心得体会

完成了本实验的要求，我有以下感悟：

- 通过 STL 容器组合（map + vector + set）可以高效完成文本索引任务；
- 多从利用网络搜索从标准库中寻找合适的算法与数据结构，能大大简化实现过程，提高代码效率与可读性；

加分项

本次实验除完成基本功能外，还在以下方面进行了扩展与优化，我认为可以视作加分项：

- 完成了本实验的要求和选做的函数。
- 实现从文件中读取名字，同时对多处使用的标准库函数使用方式标注信息来源；
- 采用自动化测试（Google Test）：使用 Google Test 编写系统化测试用例，实现了对各函数逻辑的自动验证，体现了工程化测试思维；
- 接口与实现分离设计：将函数的声明与定义分别放置于.h 和.cpp 文件中，遵循模块化与可维护性原则。

附录：全部代码

Listing 4: fileprocessing.h

```cpp
#ifndef FILEPROCESSING_H
#define FILEPROCESSING_H
#include <string>
#include <vector>
#include <iostream>
#include <map>
#include <set>
#include <utils.h>

class FileProcessing {
public:
    FileProcessing(const std::string& filename);
    ~FileProcessing() = default;
    FileProcessing(const FileProcessing& other);
    FileProcessing& operator=(const FileProcessing& other);
    bool IsWordAppear(const std::string& word);
    int CountWordOccurrences(const std::string& word);
    void AddNewWord(const std::string& word, int row, int col);
    void SaveToFile(const std::string& filename);
    void DeleteWord(const std::string& word);
    void DisplayWords();
    void PrintWordOccur();
    void PrintWordsAscend();
    void PrintWordCount();
    void PrintTopWords(int n);
private:
    std::map<std::string, std::vector<std::pair<int, int>>> wordMap;
    std::set<std::string> wordSet;
};
#endif
```

Listing 5: utils.h

```cpp
#ifndef UTILS_H
#define UTILS_H
#include <string>
#include <vector>
#include <utility>
#include <map>
namespace utils {
```

```
    void File2WordMap(const std::string& filename,
                      std::map<std::string, std::vector<std::pair<int, int>>>& wordMap);
    bool CheckViable(int row, int col,
                     std::map<std::string, std::vector<std::pair<int, int>>>& wordMap);
    std::vector<std::pair<std::string, int>> getCount(
        std::map<std::string, std::vector<std::pair<int, int>>>& wordMap);
}
#endif
```

Listing 6: utils.cpp

```
#include "utils.h"
#include <iostream>
#include <vector>
#include <string>
#include <map>
#include <set>
#include <fstream>
#include <sstream>

namespace utils {
    void File2WordMap(const std::string& filename,
                      std::map<std::string, std::vector<std::pair<int, int>>>& wordMap) {
        std::ifstream input(filename);
        if (!input.is_open()) {
            std::cerr << "Failed to open file: " << filename << std::endl;
            return;
        }
        std::string line;
        int lineNumber = 0;
        while (std::getline(input, line)) {
            lineNumber++;
            std::istringstream iss(line);
            std::string word;
            int columnNumber = 0;
            while (iss >> word) {
                columnNumber++;
                wordMap[word].emplace_back(lineNumber, columnNumber);
            }
        }
    }

    bool CheckViable(int row, int col,
                     std::map<std::string, std::vector<std::pair<int, int>>>& wordMap) {
```

```cpp
        if (row < 1 || col < 1) { return false; }
        if (wordMap.empty()) { return true; }
        std::pair<int, int> pos = std::make_pair(row, col);
        for (auto word : wordMap) {
            for (auto position : word.second) {
                if (pos == position) { return false; }
            }
        }
        for (auto word : wordMap) {
            for (auto position : word.second) {
                if (position.first == row && position.second + 1 == col) { return true; }
            }
        }
        if (col == 1) {
            for (auto word : wordMap) {
                for (auto position : word.second) {
                    if (position.first == row - 1) { return true; }
                }
            }
        }
        return false;
    }

    std::vector<std::pair<std::string, int>> getCount(
        std::map<std::string, std::vector<std::pair<int, int>>>& wordMap) {
        std::vector<std::pair<std::string, int>> wordCount;
        for (auto entry : wordMap) {
            std::pair<std::string, int> pair =
                std::make_pair(entry.first, static_cast<int>(entry.second.size()));
            wordCount.push_back(pair);
        }
        std::sort(wordCount.begin(), wordCount.end(),
                  [](const auto& a, const auto& b) { return a.second < b.second; });
        return wordCount;
    }
}
```

Listing 7: fileprocessing.cpp

```cpp
#include "utils.h"
#include "fileprocessing.h"
#include <iostream>
#include <vector>
#include <string>
```

```cpp
#include <map>
#include <set>
#include <fstream>
#include <algorithm>

FileProcessing::FileProcessing(const std::string& filename) {
    std::map<std::string, std::vector<std::pair<int, int>>> wordMap;
    utils::File2WordMap(filename, wordMap);
    this->wordMap = wordMap;
    for (const auto& entry : wordMap) {
        wordSet.insert(entry.first);
    }
}

FileProcessing::FileProcessing(const FileProcessing& other) {
    this->wordMap = other.wordMap;
    this->wordSet = other.wordSet;
}

FileProcessing& FileProcessing::operator=(const FileProcessing& other) {
    if (this != &other) {
        this->wordMap = other.wordMap;
        this->wordSet = other.wordSet;
    }
    return *this;
}

bool FileProcessing::IsWordAppear(const std::string& word) {
    return wordMap.find(word) != wordMap.end();
}

int FileProcessing::CountWordOccurrences(const std::string& word) {
    auto it = wordMap.find(word);
    if (it != wordMap.end()) { return static_cast<int>(it->second.size()); }
    return 0;
}

void FileProcessing::AddNewWord(const std::string& word, int row, int col) {
    if (!utils::CheckViable(row, col, this->wordMap)) {
        std::cerr << "输入位置非法" << std::endl;
        return;
    }
    wordMap[word].emplace_back(row, col);
}
```

```cpp

void FileProcessing::SaveToFile(const std::string& filename) {
    std::ofstream output(filename);
    if (!output.is_open()) {
        std::cerr << "无法打开文件进行写入: " << filename << std::endl;
        return;
    }
    for (const auto& entry : wordMap) {
        const std::string& word = entry.first;
        const auto& positions = entry.second;
        for (const auto& pos : positions) {
            output << word << " " << pos.first << " " << pos.second << std::endl;
        }
    }
    output.close();
}

void FileProcessing::DeleteWord(const std::string& word) {
    auto it = wordMap.find(word);
    if (it == wordMap.end()) { return; }

    std::map<int, std::vector<int>> rowToDeleteCols;
    for (const auto& pos : it->second) {
        rowToDeleteCols[pos.first].push_back(pos.second);
    }
    for (auto& entry : rowToDeleteCols) {
        std::sort(entry.second.begin(), entry.second.end());
    }

    std::map<std::string, std::vector<std::pair<int, int>>> newWordMap;
    for (const auto& entry : wordMap) {
        const std::string& curWord = entry.first;
        const auto& positions = entry.second;
        std::vector<std::pair<int, int>> newPositions;
        newPositions.reserve(positions.size());

        for (const auto& pos : positions) {
            auto rowIt = rowToDeleteCols.find(pos.first);
            if (rowIt != rowToDeleteCols.end()) {
                const auto& delCols = rowIt->second;
                if (curWord == word &&
                    std::binary_search(delCols.begin(), delCols.end(), pos.second)) {
                    continue;
                }
```

```
                int shift = std::lower_bound(delCols.begin(), delCols.end(), pos.second) - delCols.begin();
                newPositions.emplace_back(pos.first, pos.second - shift);
            } else {
                newPositions.emplace_back(pos);
            }
        }
        if (!newPositions.empty()) { newWordMap[curWord] = std::move(newPositions); }
    }

    wordMap = std::move(newWordMap);
    wordSet.clear();
    for (const auto& entry : wordMap) {
        wordSet.insert(entry.first);
    }
}

void FileProcessing::DisplayWords() {
    for (const auto& entry : wordMap) {
        std::cout << entry.first << ":";
        for (const auto& pos : entry.second) {
            std::cout << "(" << pos.first << ", " << pos.second << ")";
        }
        std::cout << std::endl;
    }
}

void FileProcessing::PrintWordOccur() {
    std::vector<std::pair<std::string, int>> wordCount = utils::getCount(wordMap);
    for (auto& entry : wordCount) {
        std::cout << entry.first << " ";
    }
    std::cout << std::endl;
}

void FileProcessing::PrintWordCount() {
    std::vector<std::pair<std::string, int>> wordCount = utils::getCount(wordMap);
    for (auto& entry : wordCount) {
        std::cout << entry.first << ':' << entry.second << ' ';
    }
    std::cout << std::endl;
}

void FileProcessing::PrintTopWords(int n) {
```

```
    std::vector<std::pair<std::string, int>> wordCount = utils::getCount(wordMap);
    n = n > wordCount.size() ? wordCount.size(): n;
    for (int i = 0; i < n; i++) {
        std::cout << wordCount[n].first << ' ';
    }
    std::cout << std::endl;
}

void FileProcessing::PrintWordsAscend() {
    std::vector<std::pair<std::string, int>> wordCount = utils::getCount(wordMap);
    std::sort(wordCount.begin(), wordCount.end(),
              [](const auto& a, const auto& b){ return a.first < b.first; });
    for (const auto& word : wordCount) {
        std::cout << word.first << ' ';
    }
    std::cout << std::endl;
}
```

注：要查看检测信息，需要文件结构满足以下结构：

```
lab8/
|- utils.h
|- utils.cpp
|- fileprocessing.h
|- fileprocessing.cpp
|- main.cpp
```

随后在终端输入

```
$ g++ -std=c++17 utils.cpp fileprocessing.cpp main.cpp -I. -lgtest -lpthread -o test_bin
$ ./test_bin
```

Listing 8: main.cpp

```
#include "fileprocessing.h"
#include <gtest/gtest.h>
#include <fstream>
#include <string>
#include <vector>

namespace {
std::string WriteFile(const std::string& filename, const std::string& content) {
    std::ofstream out(filename);
    out << content;
    out.close();
    return filename;
```

```cpp
}

std::vector<std::string> ReadLines(const std::string& filename) {
    std::ifstream in(filename);
    std::vector<std::string> lines;
    std::string line;
    while (std::getline(in, line)) {
        if (!line.empty() && line.back() == '\r') { line.pop_back(); }
        lines.push_back(line);
    }
    return lines;
}
} // namespace

TEST(FileProcessing, LoadAndCount) {
    std::string filename = WriteFile("gtest_input.txt", "apple banana apple\nbanana cherry\n");
    FileProcessing fp(filename);
    EXPECT_TRUE(fp.IsWordAppear("apple"));
    EXPECT_FALSE(fp.IsWordAppear("orange"));
    EXPECT_EQ(fp.CountWordOccurrences("apple"), 2);
    EXPECT_EQ(fp.CountWordOccurrences("banana"), 2);
    EXPECT_EQ(fp.CountWordOccurrences("cherry"), 1);
}

TEST(FileProcessing, AddNewWordAndSave) {
    std::string filename = WriteFile("gtest_input2.txt", "apple banana apple\nbanana cherry\n");
    FileProcessing fp(filename);
    fp.AddNewWord("date", 2, 3);
    fp.AddNewWord("date", 2, 5);
    EXPECT_EQ(fp.CountWordOccurrences("date"), 1);

    std::string outFile = "gtest_output2.txt";
    fp.SaveToFile(outFile);
    std::vector<std::string> lines = ReadLines(outFile);
    std::vector<std::string> expected = {
        "apple 1 1",
        "apple 1 3",
        "banana 1 2",
        "banana 2 1",
        "cherry 2 2",
        "date 2 3",
    };
    EXPECT_EQ(lines, expected);
}
```

```

TEST(FileProcessing, DeleteWordShiftsPositions) {
    std::string filename = WriteFile("gtest_input3.txt", "apple banana apple\nbanana cherry\n");
    FileProcessing fp(filename);
    fp.DeleteWord("banana");

    std::string outFile = "gtest_output3.txt";
    fp.SaveToFile(outFile);
    std::vector<std::string> lines = ReadLines(outFile);
    std::vector<std::string> expected = {
        "apple 1 1",
        "apple 1 2",
        "cherry 2 1",
    };
    EXPECT_EQ(lines, expected);
}

TEST(FileProcessing, PrintWordsAscend) {
    std::string filename = WriteFile("gtest_input4.txt", "banana apple cherry\n");
    FileProcessing fp(filename);
    testing::internal::CaptureStdout();
    fp.PrintWordsAscend();
    std::string output = testing::internal::GetCapturedStdout();
    EXPECT_EQ("apple banana cherry \n", output);
}

int main(int argc, char **argv) {
    ::testing::InitGoogleTest(&argc, argv);
    return RUN_ALL_TESTS();
}
```